Avaliação Continuada 04

Questão 3 (Programação Dinâmica)

(Impressão nítida) Desejamos determinar em quais palavras realizar quebras de linha, de modo a não sobrar muito espaço no final das linhas, exceto na última linha, que pode terminar com muitos espaços. Quando quebramos a linha em uma palavra, essa palavra inicia a linha seguinte. Portanto, uma solução consiste em um conjunto de índices de palavras nas quais realizamos quebras de linha. O custo de uma solução é a soma dos quadrados dos espaços que sobram no final de cada linha, exceto na última linha, cujo custo é sempre zero.

Como entrada, recebemos um vetor $L[1..n]$, em que $L[i]$ representa o número de caracteres da palavra i, e um inteiro W, que representa o número máximo de caracteres que podem ser colocados em uma linha.

Considere que o tamanho de toda palavra é menor ou igual a W, isto é, $L[i] \leq W$, para todo $i \in \{1, \ldots, n\}$.

Seja $M[i, j]$ o número de caracteres utilizados por uma linha que começa na palavra i e termina na palavra j, incluindo os espaços entre as palavras. Podemos obter $M[i, j]$ somando os tamanhos das palavras de i até j, mais os $j - i$ espaços necessários para separar estas palavras. Temos:

$$M[i, j] = \sum_{k=i}^{j} L[k] + (j - i).$$

O custo de uma linha que começa na palavra i e termina na palavra j é dado por:

$$C[i, j] = \begin{cases} 0, & \text{se } j = n, \\ (W - M[i, j])^2, & \text{caso contrário.} \end{cases}$$

Exemplo:

```
Texto:       abcde fghijk lm nopqr stuv x w yz abcde fg hijkl
Índices:     1     2      3  4     5    6 7 8  9     10 11
Tamanhos:    5     6      2  5     4    1 1 2  5     2  5

Tamanho máximo de uma linha: W = 12.
```

Qual é o custo da solução que realiza quebras de linha antes das palavras 3, 5, 8 e 11?

Texto	Índices	Tamanho da linha	Custo
abcde fghijk	$1, 2$	$5 + 6 + 1 = 12$	$(12 - 12)^2 = 0$

Texto	Índices	Tamanho da linha	Custo
lm nopqr	$3, 4$	$2 + 5 + 1 = 8$	$(12 - 8)^2 = 16$
stuv x w	$5, 6, 7$	$4 + 1 + 1 + 1 + 1 = 8$	$(12 - 8)^2 = 16$
yz abcde fg	$8, 9, 10$	$2 + 5 + 2 + 2 = 11$	$(12 - 11)^2 = 1$
hijkl	11	5	0, última linha

Portanto, o custo total dessa solução é $0 + 16 + 16 + 1 = \boxed{33}$.

Podemos preencher a matriz M em tempo $O(n^2)$ utilizando o seguinte algoritmo:

```
para i de 1 até n
    M[i,i] = L[i]
    para j de i+1 até n
        M[i,j] = M[i,j-1] + L[j] + 1
```

O algoritmo recursivo de backtracking abaixo encontra uma solução ótima testando todas as possibilidades para a próxima quebra de linha e resolvendo recursivamente o subproblema a partir dessa quebra. Porém, ele possui complexidade de tempo exponencial.

```
Algoritmo impressão_nítida(M[1..n,1..n], W, i):

Entrada:
    M[a,b]: tamanho da linha que começa na palavra a
            e termina na palavra b.
    W: tamanho máximo de uma linha.
    i: índice da primeira palavra do subproblema.

Saída:
    Tupla (sol, custo), em que sol é o conjunto de índices
    das quebras de linha de uma solução ótima e custo é
    o custo dessa solução.

01  se M[i,n] <= W
02      retorne ({}, 0)

03  menor_custo = infinito
04  para j de i+1 até n
05      se M[i,j-1] > W então break
06      (sub_sol, sub_custo) = impressão_nítida(M,W,j)
07      custo = (W - M[i,j-1])^2 + sub_custo
08      se custo < menor_custo
09          menor_custo = custo
10          melhor_solucao = sub_sol U {j}
11  retorne (melhor_solucao, menor_custo)
```

Com base no algoritmo recursivo de backtracking acima, projete um algoritmo de **programação dinâmica** de tempo polinomial para resolver o mesmo problema.

a. Qual é o número de subsoluções armazenadas na tabela de programação dinâmica? Justifique.
b. Como é feito o preenchimento do caso base?
c. Qual é a ordem de preenchimento da tabela? Argumente que essa ordem não acessa elementos ainda não preenchidos.

d. Forneça o pseudocódigo do algoritmo.
e. Qual é a complexidade de tempo do algoritmo?

Observação: A linha 10 do algoritmo recursivo realiza uma cópia da lista `sub_sol`, de modo que seu custo depende do número de elementos dessa lista. Entretanto, é possível recuperar a solução ótima armazenando apenas o índice j da próxima quebra de linha para cada posição inicial i. Assim, para simplificar a análise, considere que a execução da linha 10 tem tempo constante.
